

Выпускная квалификационная работа
(Дипломная работа)

На тему: Разработка веб-информационной системы агрегации и
анализа вакансий из нескольких источников с поиском,
фильтрацией и уведомлениями на почту (на примере ФГБОУ
ВО «Российский экономический университет им. Г. В.
Плеханова»)

ЧИСТОТИНА МИХАИЛА ЮРЬЕВИЧА
Студента 4 курса группы П50-6-22

по специальности 09.02.07 «Информационные системы и программирование»
для присвоения квалификации: программист
Форма обучения: очная

Руководитель: _____ / Юшина Дарья Денисовна /
(подпись)
« ____ » _____ 2026 г.

Студент _____ / Чистотин Михаил Юрьевич /
(подпись)
« ____ » _____ 2026 г.

Допущен к защите

Приказ от «06» апреля 2026 г. № 18.01-17-156

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1. ОБЩАЯ ЧАСТЬ.....	5
1.1. Цель разработки	5
1.2. Средства разработки	5
2. СПЕЦИАЛЬНАЯ ЧАСТЬ	8
2.1. Постановка задачи	8
2.1.1. Входные данные предметной области.....	8
2.1.2. Выходные данные предметной области	9
2.1.3. Подробные требования к проекту.....	10
2.2. Внешняя спецификация.....	11
2.2.1. Описание задачи	11
2.2.2. Входные и выходные данные пользовательского интерфейса.....	15
2.2.3. Методы.....	15
2.2.4. Тесты.....	17
2.2.5. Контроль целостности данных.....	17
2.3. Проектирование.....	20
2.3.1. Схема архитектуры приложения.....	20
2.3.2. Логическая модель базы данных	20
2.3.3. Физическая модель базы данных	23
2.3.4. Структурная схема программной реализации	25
2.3.5. Функциональная схема	27
2.3.6. Схема тестирования	29
2.3.7. Схема пользовательского интерфейса	29
2.3.8. Диаграмма классов	31
2.4. Результат работы программы	31

3. ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ	33
3.1. Инструментальные средства разработки	33
3.2. Отладка программы	33
3.3. Защитное программирование	34
3.4. Характеристики программы.....	36
ЗАКЛЮЧЕНИЕ	39
ПРИЛОЖЕНИЕ А Текст программы.	
ПРИЛОЖЕНИЕ Б Скрипт базы данных.	
ПРИЛОЖЕНИЕ В Руководство пользователя	
ПРИЛОЖЕНИЕ Г Сценарий и результаты тестовых испытаний	
ПРИЛОЖЕНИЕ Д Результаты анализа предметной области	
ПРИЛОЖЕНИЕ Е Словарь данных	
ПРИЛОЖЕНИЕ Ж Диаграмма классов	

ВВЕДЕНИЕ

В течение длительного времени подбор персонала у части работодателей и соискателей фактически ведётся вручную: вакансии и отклики разбросаны по разным сайтам и мессенджерам, статусы кандидатов фиксируются в таблицах, а уведомления отправляются по отдельным каналам. Такой порядок работы увеличивает трудозатраты и повышает риск ошибок и потери информации между этапами отклика, собеседования и принятия решения.

При анализе предметной области выявлена потребность в автоматизированном программном продукте, который обеспечивает централизованное хранение и отображение вакансий, поиск и фильтрацию по заданным критериям, учёт взаимодействия соискателя и работодателя (в том числе отклики, сообщения, календарь и напоминания), а также пополнение каталога данными из нескольких внешних источников с разграничением записей по источнику.

Целью выпускной квалификационной работы является разработка такой web-информационной системы и описание технологии её создания, задачами – проектирование программных средств, реализация ключевых модулей, тестирование и подготовка материалов для внедрения и эксплуатации.

1. ОБЩАЯ ЧАСТЬ

1.1. Цель разработки

Цель разработки – создать web-информационную систему агрегации и анализа вакансий из нескольких источников с поиском, фильтрацией и почтовыми уведомлениями, чтобы снизить трудозатраты и ускорить работу соискателей и работодателей за счёт единого каталога и согласованных процедур с данными.

1.2. Средства разработки

Для разработки web-информационной системы и сопутствующей инфраструктуры использовались технические средства, сведения о которых приведены в таблице 1.

Таблица 1 – Технические средства для разработки

№	Тип оборудования	Наименование и основные характеристики
1	2	3
1	Персональная электронная вычислительная машина (ноутбук)	Lenovo IdeaPad 5 (14IIL05)
1.1	Операционная система	Microsoft Windows 10 Home (лицензированная копия)
1.2	Диагональ экрана	14 дюймов
1.3	Разрешение экрана	Full HD (1920 на 1080 пикселей)
1.4	Тип матрицы	IPS
1.5	Центральный процессор	Intel Core i5-1035G1
1.6	Количество физических ядер	4
1.7	Количество логических процессоров	8
1.8	Объём оперативной памяти	16 Гбайт DDR4
1.9	Накопитель	SSD, 512 Гбайт
1.10	Графический адаптер	Intel UHD Graphics (встроенный)

Примечание. При отладке и демонстрации дополнительно использовалась иная вычислительная техника (персональный компьютер)

Минимальные и рекомендуемые технические средства для эксплуатации готового программного продукта (рабочее место пользователя web-системы) приведены в таблице 2.

Таблица 2 – Требования к техническим средствам эксплуатации

№	Параметр	Минимальные значения	Рекомендуемые значения
1	2	3	4
1	Тип устройства	Персональный компьютер или ноутбук	Ноутбук или моноблок
2	Операционная система	Microsoft Windows 10 (64-разрядная) или совместимая ОС с современным браузером	Microsoft Windows 10 или 11 (64-разрядная)
3	Центральный процессор	Двухъядерный процессор с тактовой частотой не ниже 1,6 ГГц	Четырёхъядерный процессор и выше
4	Оперативная память	не менее 4 Гбайт	не менее 8 Гбайт
5	Накопитель	не менее 2 Гбайт свободного места для установки компонентов и хранения служебных данных	SSD, не менее 10 Гбайт свободного места
6	Дисплей	Диагональ не менее 12 дюймов, разрешение не ниже 1366 на 768 пикселей	Диагональ не менее 14 дюймов, разрешение не ниже 1920 на 1080 пикселей
7	Сеть	Доступ в Интернет для работы внешних интеграций и доставки уведомлений	Широкополосный доступ в Интернет
8	Устройства ввода	Клавиатура и устройство указания, совместимые с используемой ОС	Полноразмерная клавиатура, оптическая мышь

Для разработки использовались программные средства, перечисленные в таблице 3.

Таблица 3 – Программные средства разработки

№	Тип средства	Название средства	Назначение
1	2	3	4
1	Средство разработки	Visual Studio Code	Редактирование кода и отладка
2	Язык программирования	Python[8]	Реализация серверной части и сценариев
3	Текстовый редактор	Microsoft Word версия 2604 сборка 16.0.19929.20086	Разработка документации, формирование отчётных документов по шаблонам
4	Фреймворк	Django[3]	web-приложение, ORM, шаблоны, администрирование
5	Библиотека API	Django REST framework[4]	Построение REST API

№	Тип средства	Название средства	Назначение
1	2	3	4
6	Фоновые задачи	Celery[5], Redis[6]	Периодические и отложенные операции, очередь
7	СУБД	SQLite[7]	Хранение данных
8	Документация API	Swagger[15] и ReDoc[16] (на базе OpenAPI[13])	Описание и проверка методов API
9	Контроль версий	Git[14]	Учёт изменений исходного кода
10	Проверка сборки проекта	GitLab CI[21]	Реализация Pipelines
11	Контейнеризация	Docker[22]	Запуск воркера Celery
12	Тестирование HTTP[17]	Postman[20]	Проверка запросов к API

2. СПЕЦИАЛЬНАЯ ЧАСТЬ

2.1. Постановка задачи

Требуется разработать web-информационную систему агрегации и анализа вакансий из нескольких источников с поиском, расширенной фильтрацией и уведомлениями по e-mail (допускается Telegram). Система должна обеспечивать единый каталог, различение по источнику, актуализацию через внешние сервисы, взаимодействие соискателя и работодателя при вакансиях на сайте (отклики, статусы, переписка, собеседования), ролевой доступ, модерацию контента и учёт записей – в объёме реализации.

2.1.1. Входные данные предметной области

Входные данные – сведения и команды от пользователей, администраторов и внешних API.

- Учётные данные: регистрация (логин, e-mail, пароль и подтверждение, ФИО); вход (логин, пароль).
- Профиль соискателя: контакты, демография и навыки; зарплатные ожидания; «о себе», должность, ссылки; место (метро или адрес); аватар, файл резюме; образование, доп. образование, опыт.
- Профиль работодателя: организация, контакты, Telegram, аватар, логотип.
- Локальная вакансия: поля формы (регион, зарплата, формат и график, тип занятости, тексты, адрес или координаты, контакт, метро, даты, признаки активности и пр.).
- Найм: отклик (вакансия–пользователь, письмо, статус); собеседование (дата и время, место или ссылка, заметки, статус); чат (сообщения, вложения); календарь (дата, время, текст, цвет заметки).

- Поиск и фильтры: запрос, параметры (регион, зарплата, формат, график, занятость, источник), сохранённые пресеты.
- Интерфейс: тема (светлая/тёмная).
- Админ/модерация: управление ролями и учётными записями, журнал, резервное копирование/восстановление, жалобы и решения (статусы, скрытие/восстановление) – по реализации.
- Внешние источники: ответы API HeadHunter[9] и opendata.trudvsem.ru[10]; прочие данные для обогащения (в т.ч. рейтинги).
- Конфигурация: ключи API, интервалы синхронизации, базовый URL, прочие параметры среды.
- Журнал API: роль, метод, путь, операция, успех, код ответа, данные до или после, время.

2.1.2. Выходные данные предметной области

Выходные данные – то, что система показывает, отдаёт пользователю или передаёт по каналам после обработки.

- Каталог и карточка: список с пагинацией, фильтрами и сортировкой; на карточке – полное описание, контакты, ссылка на источник; при необходимости карта, маршрут и рейтинг; источник: НН, «Работа России», сайт.
- Соискатель: профиль, резюме, отклики и статусы, избранное, просмотры, аналитика (и проверка резюме – если подключено), экспорт при реализации.
- Работодатель: профиль, список вакансий, отклики, статусы, собеседования, чат, аналитика, выгрузка откликов.
- Коммуникации и календарь: сообщения, прочтение, вложения; собеседования и заметки.

- Уведомления: e-mail по правилам системы; при реализации – Telegram.
- Админ и модерация: сводки; журнал; список резервных копий; экраны жалоб и решений.
- API: ответы REST (JSON); Swagger или ReDoc для уполномоченных ролей.
- Интерфейс: тема, сообщения об ошибках или успехе, подсказки.

2.1.3. Подробные требования к проекту

Общие: тема – агрегация и анализ из нескольких источников, поиск, фильтры, e-mail-уведомления, сценарии соискатель–работодатель; изменение данных только по ролям; каталог – поиск, многокритериальные фильтры, постраничный вывод.

Безопасность и данные: аутентификация, ролевой доступ, CSRF для форм, хеширование паролей (Django), персональные данные с учётом 152-ФЗ[2]; реляционное хранение (SQLite) с ключами, связями и уникальностью по предметной области; валидация на сервере (клиент – дополнительно); импорт, актуализация и рассылки – в фоне (Celery), без блокировки HTTP.

Функциональный состав (реализация): регистрация, вход, выход, сессии; роли (соискатель, работодатель, администратор, модератор); каталог с фильтрами и источником; импорт, обновление минимум из НН и «Работа России»; актуализация внешних записей (скрытие, удаление локально по правилам); локальные вакансии (создание, редактирование, снятие с публикации); отклик с ограничением «один пользователь – одна вакансия» и правилами по недопустимым сценариям; чат с ограничениями вложений; собеседования, календарь, напоминания; уведомления e-mail и Telegram по событиям; закладки, просмотры, аналитика (в объёме реализации); доп. сведения о работодателе (рейтинги – где поддерживается); администратор: пользователи и роли, журнал, резервное копирование и восстановление БД;

модератор: жалобы, скрывание, восстановление с учётом статусов и отчётности; REST (DRF); Swagger и ReDoc с ограничением по ролям; фоновая очередь (Redis), планировщик (Celery Beat), согласованный доступ при пакетных операциях.

Комплект сдачи включает: пояснительную записку; машинный носитель или иное представление программного продукта; приложение А «Текст программы»; приложение Б «Скрипт базы данных»; приложение В «Руководство пользователя»; приложение Г «Сценарий и результаты тестовых испытаний»; приложение Д «Результаты анализа предметной области»; приложение Е «Словарь данных»; приложение Ж «Диаграмма классов».

2.2. Внешняя спецификация

2.2.1. Описание задачи

В рамках дипломной работы разрабатывается web-информационная система агрегации и анализа вакансий из нескольких источников с поиском, фильтрацией и уведомлениями на электронную почту. Система реализуется как клиент–серверное приложение с использованием фреймворка Django, хранением данных в реляционной базе, программным интерфейсом на базе Django REST Framework и фоновыми задачами для периодического обмена с внешними источниками и доставки уведомлений. Пользователям предоставляются роли соискателя, работодателя, администратора и модератора; внешние источники вакансий включают, как минимум, HeadHunter и портал открытых данных «Работа России», а также вакансии, создаваемые на сайте.

Для формализации предметной области и обоснования состава функций выполнен анализ процесса подбора персонала в нотации IDEF0[19]: построены модели состояния «как есть» (AS-IS) до внедрения

системы и «как будет» (ТО-ВЕ) после внедрения. В данном пункте приведены контекстные диаграммы верхнего уровня А0 для указанных состояний и краткое сравнение процессов. Декомпозиции функций, ICOM-таблицы по элементам модели и пояснения к ним с целью соблюдения методических требований к полноте моделирования вынесены в приложение Д «Результаты анализа предметной области» и не дублируются в основном тексте записки.

AS-IS – процесс до внедрения системы.

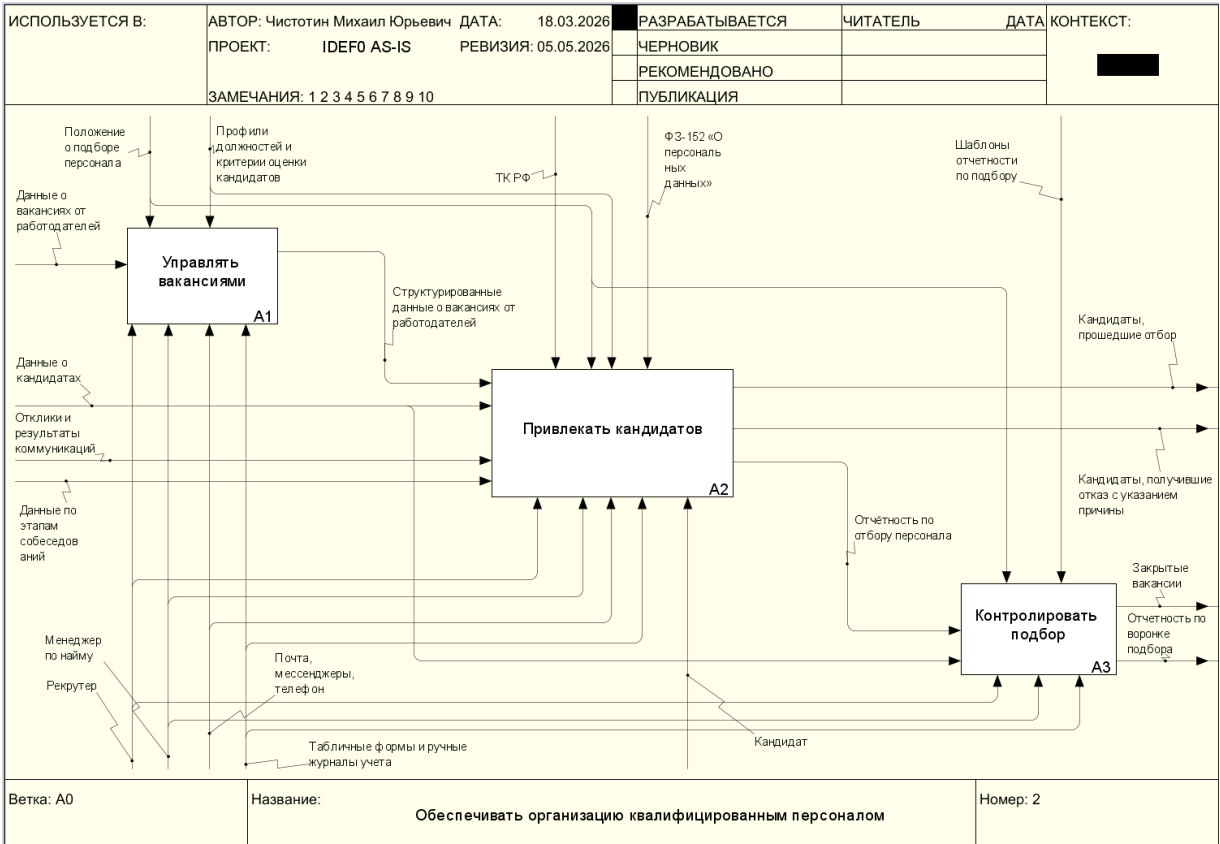


Рисунок 1 – Контекстная диаграмма А0 AS-IS

На рисунке 1 приведена контекстная диаграмма А0 в состоянии AS-IS. Процесс в целом может быть назван «обеспечение организации персоналом»: входами являются сведения о вакансиях от работодателей, данные о кандидатах, отклики и результаты взаимодействий, сведения по этапам собеседований. Управление осуществляется в рамках трудового

законодательства, требований по персональным данным, локальных положений о подборе, профилей должностей и критериев оценки, а также принятых форм отчётности. Механизмами выступают рекрутер, менеджер по найму, кандидат, электронная почта, мессенджеры, телефон, табличные формы и ручные журналы. Выходами являются закрытые вакансии, кандидаты, прошедшие отбор, кандидаты с отказом и отчёты по воронке. ICOM-описание функций AS-IS по уровням декомпозиции приведено в таблице 1 приложения Д.

ТО-ВЕ – процесс после внедрения системы.

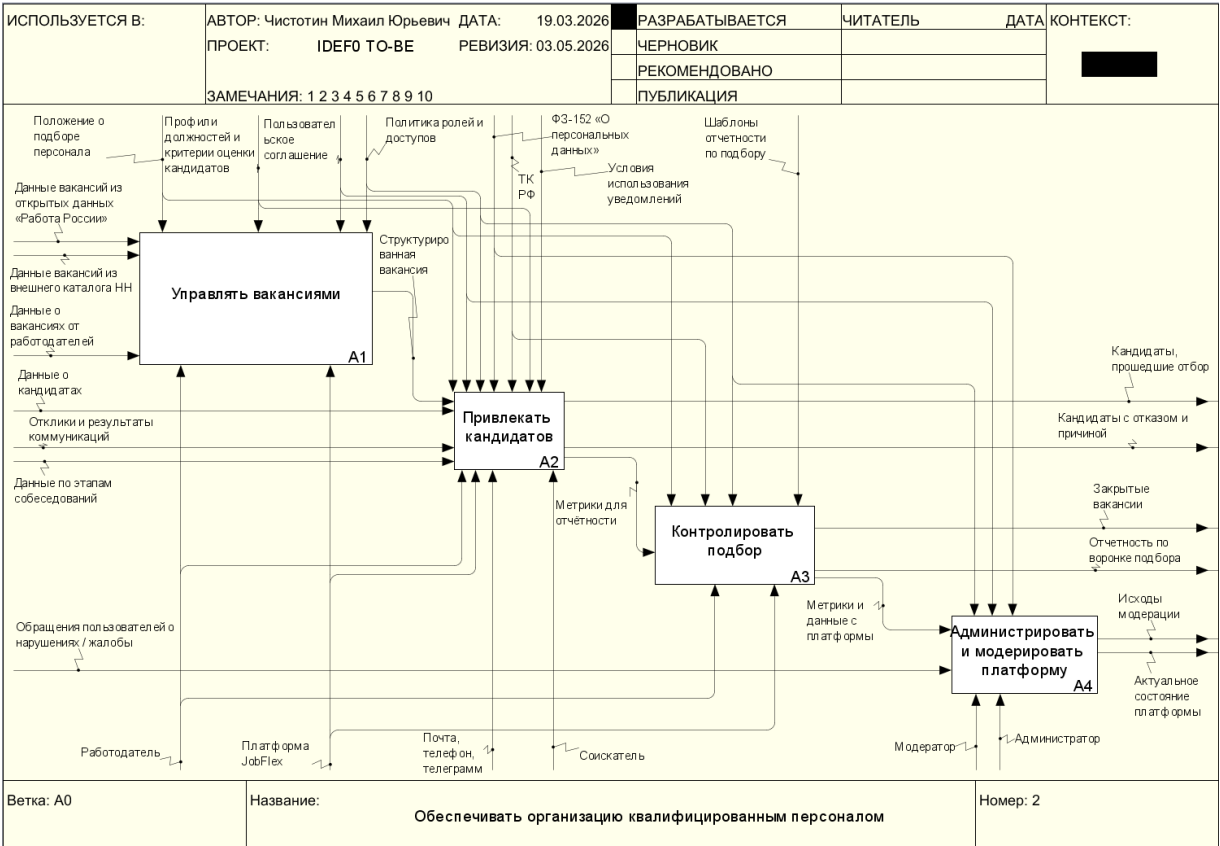


Рисунок 2 – Контекстная диаграмма A0 TO-VE

На рисунке 2 приведена контекстная диаграмма A0 для состояния ТО-ВЕ. Механизмы ручного учёта в типовых операциях заменяются разрабатываемой платформой; в управление добавляются пользовательское соглашение, правила уведомлений и ролевой доступ. Механизмами

выступают соискатель, работодатель, администратор, модератор, контактные данные пользователей и сама платформа. Выходы дополняются актуальным состоянием системы и сводками для администратора. ICOM-описание функций TO-BE приведено в таблице 2 приложения Д.

Дополнительно для описания возможностей пользователей в терминах вариантов использования построена UML[18] диаграмма прецедентов. Она представлена на рисунке 3.

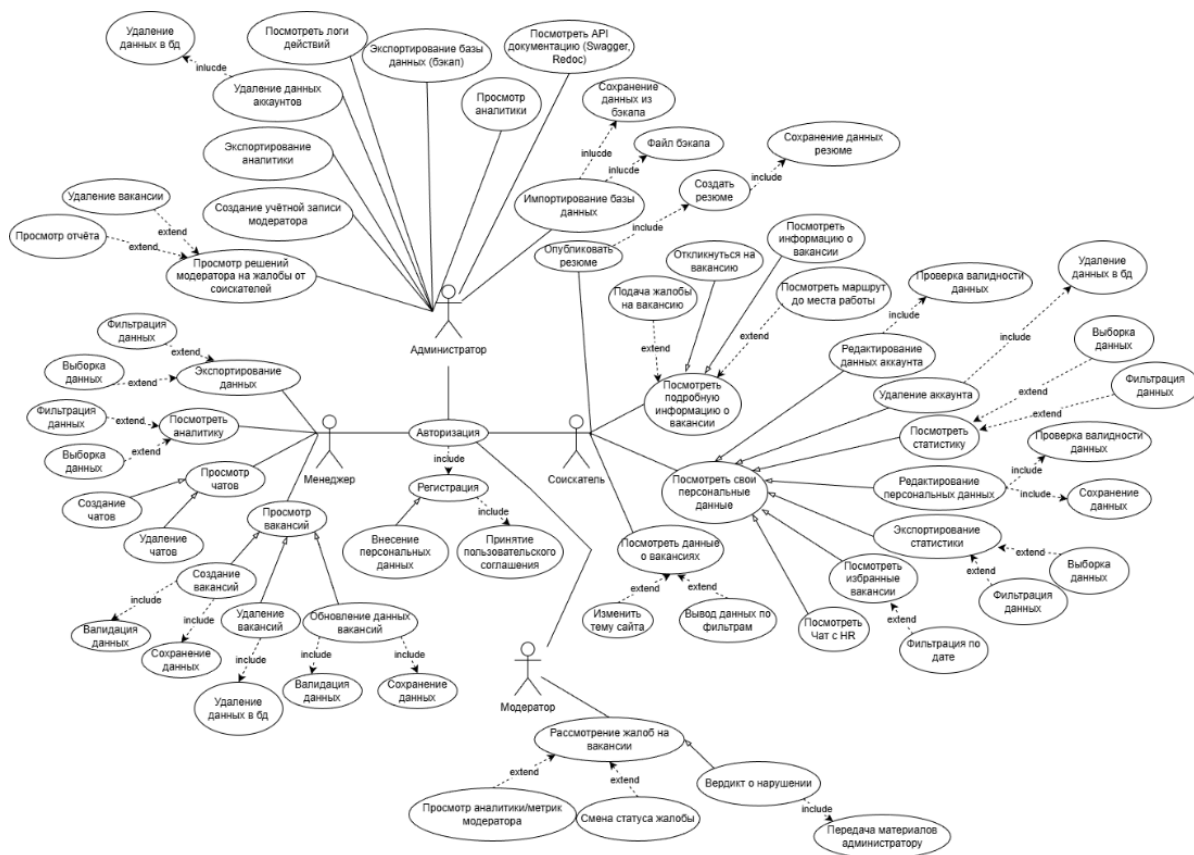


Рисунок 3 – Диаграмма прецедентов разрабатываемой системы

На диаграмме отражены прецеденты для ролей соискатель, работодатель, администратор и модератор, включая работу с каталогом (поиск, фильтры, источник вакансии), отклики, чат, календарь и собеседования, уведомления, аналитические разделы, администрирование (пользователи, журнал, резервные копии, документация API) и модерацию

(жалобы, решения по жалобам). Общим для ролей остаётся прецедент аутентификации после регистрации и принятия пользовательских условий.

2.2.2. Входные и выходные данные пользовательского интерфейса

Перечень имён полей, типов, ограничений и назначения входов и выходов пользовательского интерфейса (HTML-формы, JSON API, загрузка файлов) приведён в приложении Е «Словарь данных», таблица 1. В основной части работы приводится только классификация источников ввода (формы каталога, регистрация, профиль, вакансия, модерация, администрирование) без полного перечисления атрибутов.

2.2.3. Методы

Использование объектно-ориентированного подхода. Серверная часть платформы JobFlex реализована на языке Python с использованием фреймворка Django. Объектно-ориентированный стиль здесь проявляется прежде всего в том, что сущности предметной области (пользователь, соискатель, работодатель, вакансия, отклик, жалоба, заметка календаря и др.) описаны классами моделей, а правила хранения данных и связей между таблицами инкапсулированы на уровне ORM: прикладной код обращается к объектам и запросам к ним, не оперируя напрямую текстом SQL-запросов.

Инкапсуляция обеспечивается разделением ответственности между модулями приложений (accounts, vacancies), настройками доступа к полям и проверками в представлениях и сериализаторах, что ограничивает произвольное изменение данных извне допустимых сценариев.

Пример реализации инкапсуляции представлен на рисунке 4.

```

# -----
# Application (отклик) views
# -----

@swagger_auto_schema(method='post', operation_summary="Откликнуться на вакансию", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_apply(request):
    """Соискатель подает заявку на вакансию."""
    if not hasattr(request.user, 'applicant'):
        return JsonResponse({'error': 'not_applicant'}, status=403)
    data = request.data or {}
    vacancy_id = data.get('vacancy_id')
    cover_letter = (data.get('cover_letter') or '').strip()
    if not vacancy_id:
        return JsonResponse({'error': 'vacancy_id required'}, status=400)
    from vacancies.models import Vacancy
    from django.shortcuts import get_object_or_404
    vacancy = get_object_or_404(Vacancy, pk=vacancy_id)
    if vacancy.is_hh:
        return JsonResponse({'error': 'hh_vacancy'}, status=400)
    if not vacancy.is_active:
        return JsonResponse({'error': 'vacancy_archived'}, status=400)
    # Заблокировать подачу заявки на собственную вакансию (как работодателю, так и любому пользователю, который ее создал)
    if vacancy.created_by_id == request.user.pk:
        return JsonResponse({'error': 'own_vacancy'}, status=403)
    app, created = Application.objects.get_or_create(
        vacancy=vacancy, applicant=request.user,
        defaults={'cover_letter': cover_letter},
    )
    if not created and cover_letter:
        # allow updating cover letter if still pending
        if app.status == 'pending':
            app.cover_letter = cover_letter
            app.save(update_fields=['cover_letter'])

```

Рисунок 4 – Проверка прав и правил при отклике снаружи не обращаются в БД напрямую

Наследование используется за счёт типовой для Django схемы: модели порождены от базового класса модели, представления страниц – от классов базовых представлений или функций-обработчиков с общей логикой оформления, точки REST API построены на базовых классах Django REST Framework. Это уменьшает дублирование кода при добавлении новых сущностей и обработчиков.

Пример реализации наследования представлен на рисунке 5.

```

@swagger_auto_schema(method='post', operation_summary="Выход из системы", tags=['accounts'])
@api_view(['POST'])
@permission_classes([AllowAny])
def api_logout(request):
    try:
        auth_logout(request)
    except Exception:
        pass
    return JsonResponse({'ok': True})

```

Рисунок 5 – REST-обработчик на базе DRF

Таким образом, объектно-ориентированный подход в JobFlex опирается на модель-представление-данные в типовой для Django трактовке: объекты предметной области и их поведение сосредоточены в серверном коде, а пользовательский интерфейс выступает клиентом, использующим опубликованные методы доступа к функциям системы.

2.2.4. Тесты

Проверка программного продукта JobFlex выполнялась в соответствии с подходом к оценке качества, изложенным в ISO/IEC 25010 (модель качества систем и программных продуктов): учитывались функциональная пригодность (правильность и полнота сценариев каталога, откликов, профилей, администрирования и модерации), надёжность и устойчивость при некорректном вводе (согласование с подразделом 2.2.5), производительность и отзывчивость под нагрузкой в объёме дипломного стенда, а также безопасность в части разграничения доступа и корректности ответов API.

Использовались ручное функциональное тестирование пользовательских сценариев в браузере; автоматизированное модульное тестирование отдельных компонентов на языке Python с применением стандартной библиотеки unittest[24]; ограниченное нагрузочное тестирование сценариев обращения к web-интерфейсу с помощью Locust[23]. Результаты, формализованные тестовые наборы, схема проверки и фиксация исходов (в том числе по нагрузочному сценарию) приведены в приложении Г «Сценарий и результаты тестовых испытаний».

2.2.5. Контроль целостности данных

Целостность данных в платформе JobFlex обеспечивается на уровне ограничений моделей Django, уникальных индексов, проверок во view и REST API и корректной обработки ошибок при сохранении, изменении и

удалении записей. Ниже приведены типовые ситуации, в которых пользовательские действия могут привести к нарушению бизнес-правил или ограничений базы данных, и реакция web-приложения на такие аномалии.

В таблице 5 систематизированы ситуации по ролям: для каждой указаны характер аномалии, реакция интерфейса или API и краткое примечание.

Таблица 4 – Контроль целостности данных

№	Ситуация	Аномалия	Реакция веб-ресурса	Примечание
1	2	3	4	5
1	Изменение роли пользователя (панель администратора)	Роль вне набора admin, moderator, manager, applicant и др. Ошибка admin_set_user_role	Редирект на список пользователей с ?status=...; для недопустимой роли status=invalid_role, текст «Недопустимая роль.»	После POST change_role на admin_users
2	Удаление учётной записи (панель администратора)	Попытка удалить собственную учётную запись	Редирект с status=cannot_delete_self	Сообщение из карты status_text_map
3	Восстановление БД из резервной копии (REST, администратор)	Недопустимое имя файла (маска db_backup_YYYYMMDD_NHMMSS.sqlite3)	JsonResponse, 400, error: invalid_filename	api_admin_backup_restore
4	Восстановление БД из резервной копии (REST)	Корректное имя, файла нет в каталоге копий	404, error: not_found	
5	Удаление резервной копии (REST, администратор)	Как строки 3–4 для имени файла и наличия	400 / 404 с теми же error	api_admin_backup_delete
6	Восстановление / удаление копии (REST)	Тело запроса не является корректным JSON	400, error: invalid_json	
7	Регистрация менеджера (REST)	Дубликаты контактных данных или полей и неверный формат Telegram / не все обязательные поля	400, error: username_exists, email_exists, telegram_exists, invalid_telegram_format, missing_fields	api_register_manager

№	Ситуация	Аномалия	Реакция веб-ресурса	Примечание
1	2	3	4	5
8	Регистрация соискателя (REST)	Нарушения уникальности, телефона, возраста, ФИО-объекта, кодов допустимости полей, формата логина, Telegram без @	400, перечень кодов: username_exists, email_exists, telegram_exists, invalid_phone, too_young, invalid_birth_date, invalid_gender, invalid_city, invalid_citizenship, invalid_username, invalid_username_format, missing_fields, invalid_telegram_format	См. api_register
9	Обновление контактов пользователя (панель администратора)	Конфликт e-mail, Telegram без @, занятый Telegram	Редирект с status=email_exists / invalid_telegram / telegram_exists	Здесь используется invalid_telegram, не invalid_telegram format
10	Создание/редактирование локальной вакансии	Пустые обязательные поля; вилка зарплаты; некорректный телефон	Повторный показ формы с errors и текстами из _parse_vacancy_post	Не допускает 404 по этой причине
11	Отклик (REST), повтор	Пара «пользователь + вакансия» уже есть	200, ok: true, created: false, status: ...	
12	Отклик на внешнюю HH-вакансию	is_hh	400, error: hh_vacancy	
13	Отклик на снятую с публикации вакансию	is_active = false	400, error: vacancy_archived	
14	Отклик на «свою» вакансию	created_by_id == request.user.pk	403, error: own_vacancy	
15	Отклик не соискателя	Нет профиля Applicant	403, error: not_applicant	
16	Отклик без идентификатора вакансии	Нет vacancy_id	400, код поля vacancy_id, текст ошибки в JSON	Как в JSON ответе

2.3. Проектирование

2.3.1. Схема архитектуры приложения

Программа реализует клиент-серверную[17] модель взаимодействия: клиентский уровень (браузер) обменивается данными с сервером по протоколу HTTP. На сервере используется фреймворк Django, в котором логика разделена по принципу MVT (Model – View – Template) – варианту многослойной архитектуры

Для программного доступа к функциям системы дополнительно применяется Django REST Framework; описание REST-интерфейса доступно в Swagger или ReDoc. Данные хранятся в СУБД SQLite. Длительные и периодические операции (обновление внешних каталогов, напоминания) вынесены в фоновые задачи Celery. Общая структура компонентов приведена на рисунке 6.

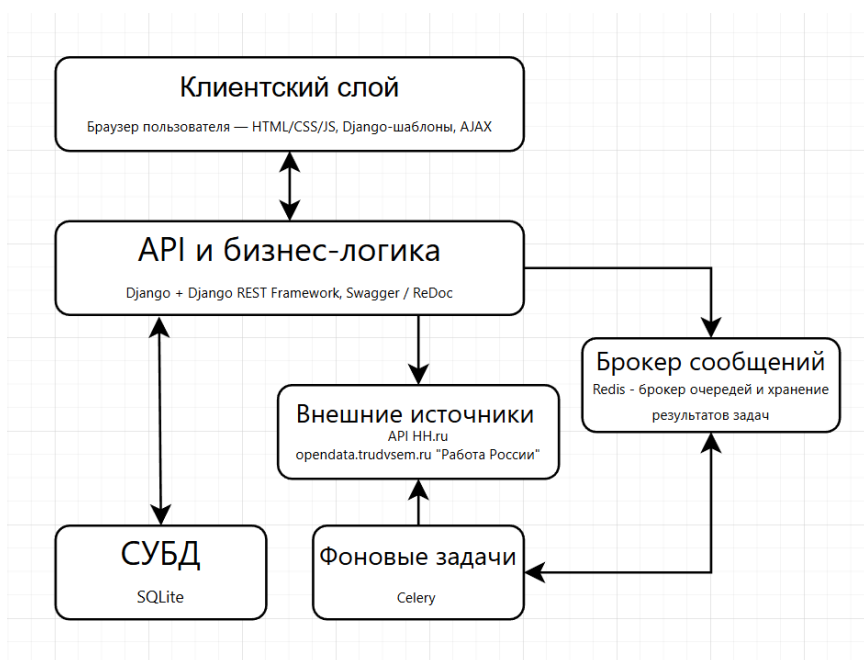


Рисунок 6 – Архитектура программного комплекса JobFlex

2.3.2. Логическая модель базы данных

На рисунке 7 показана логическая (инфологическая) модель данных JobFlex: ключевые сущности предметной области и связи между ними без

детализации физических типов и имён таблиц. Модель согласована с реляционной БД SQLite, используемой приложением Django.

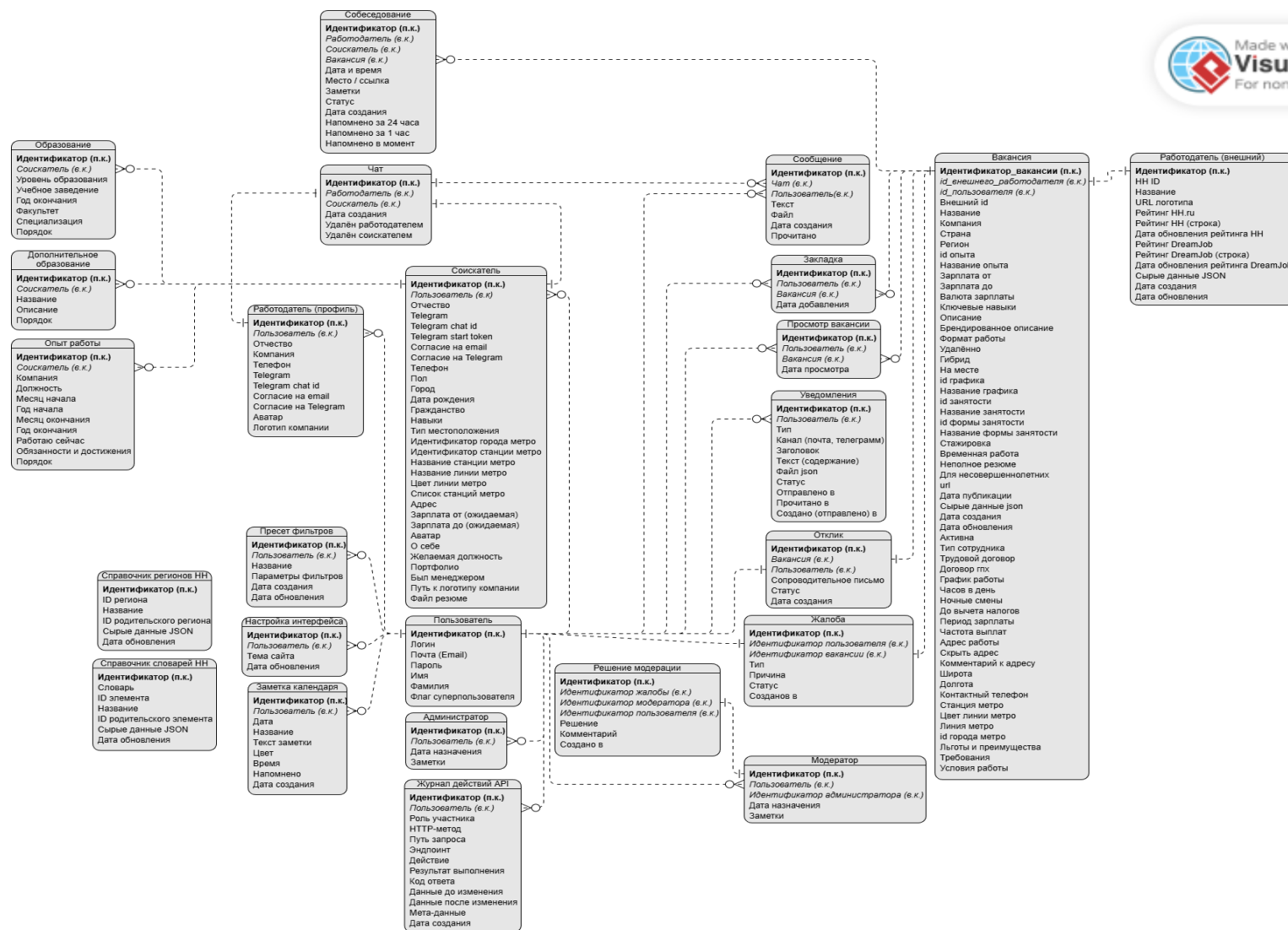


Рисунок 7 – Логическая схема базы данных JobFlex

2.3.3. Физическая модель базы данных

На рисунке 8 представлена физическая (реляционная) модель данных JobFlex, реализованная в SQLite через Django ORM. Схема показывает таблицы, ключи, связи и основные ограничения целостности.

Подробные атрибуты, типы и обязательность полей приведены в словаре данных, который вынесен в приложение Е «Словарь данных» в таблице 1.

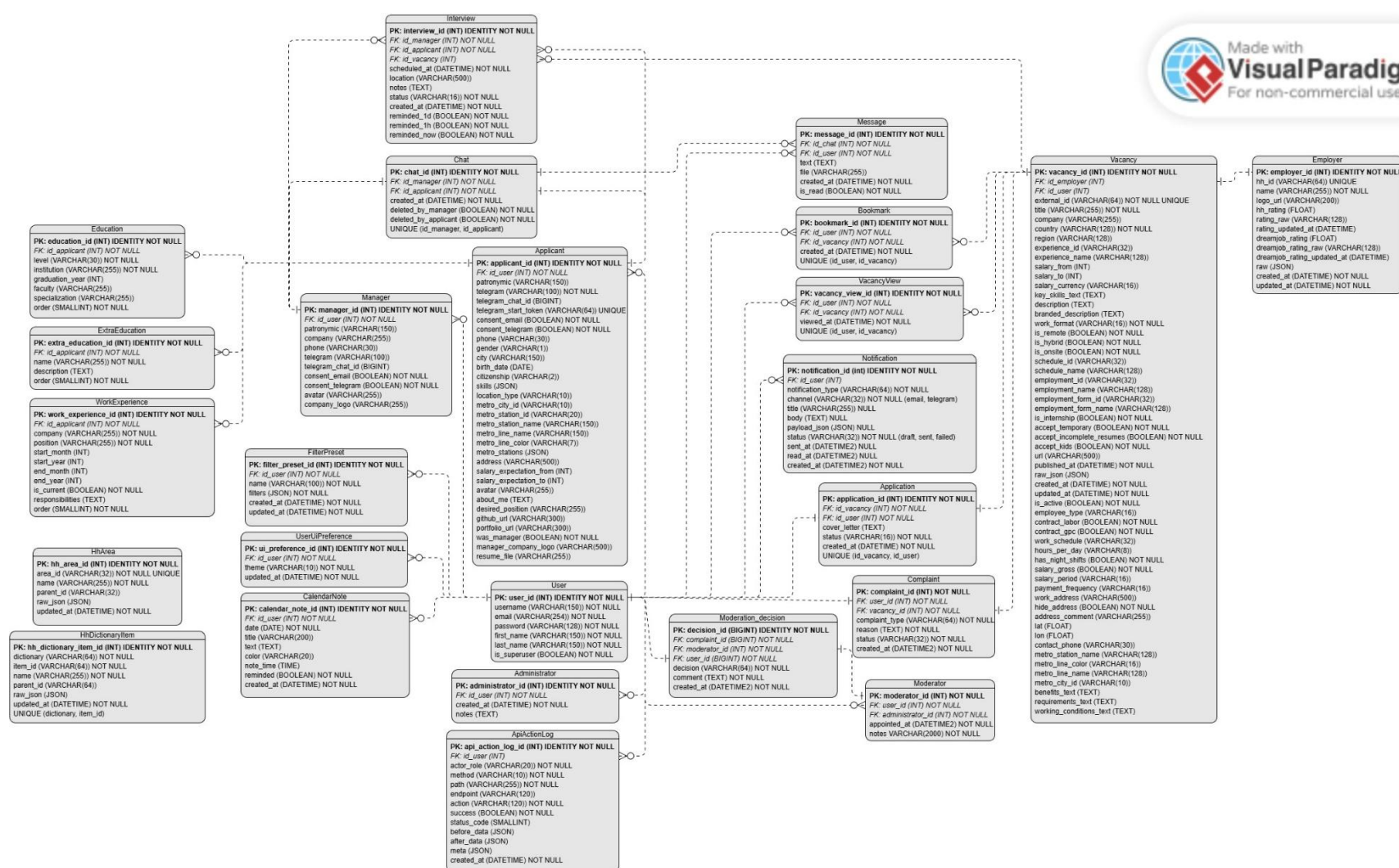


Рисунок 8 – Физическая схема базы данных

2.3.4. Структурная схема программной реализации

Структурная схема показывает организацию исходного кода и артефактов JobFlex: каталоги, границы Django-приложений и роль основных подсистем. Она дополняет архитектурную схему («клиент – сервер – фоновые задачи») переходом к репозиторию и облегчает навигацию при сопровождении и сверке с требованиями.

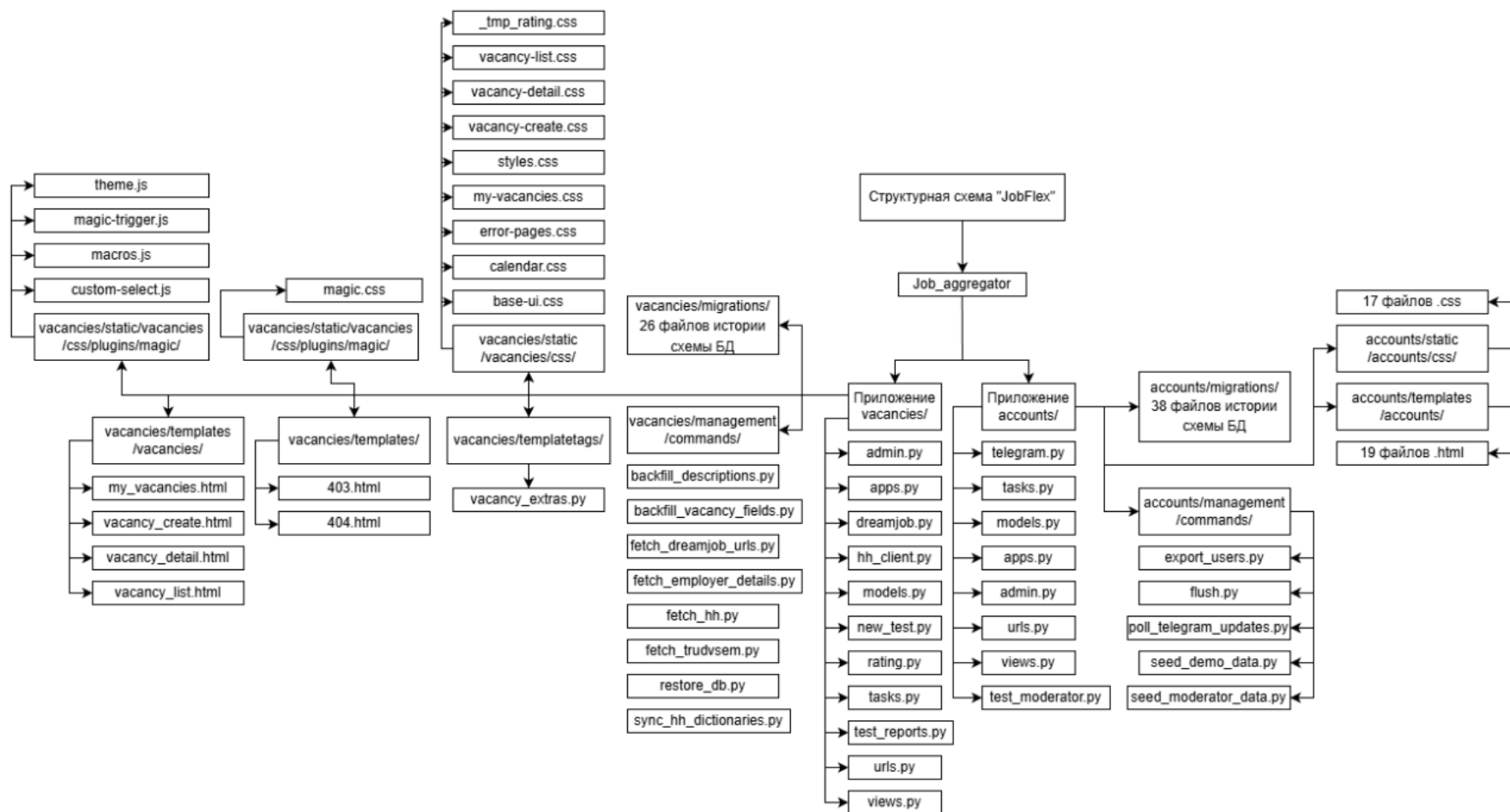


Рисунок 9 – Структурная схема web-приложения JobFlex

На рисунке 9 – единая схема репозитория: корневые артефакты (manage.py, скрипты запуска, общие шаблоны), пакет config (настройки, маршрутизация, Celery, WSGI/ASGI), каталоги media, backups, tools, plugins, а также приложения accounts и vacancies.

Для accounts раскрыты models.py, views.py, urls.py, tasks.py, telegram.py, admin.py, вспомогательные и тестовые модули (при наличии), management/commands, migrations, templates, static с ключевыми файлами.

Для vacancies – models.py, views.py, urls.py, tasks.py, модули интеграции и обработки данных (hh_client.py, dreamjob.py, rating.py и др. по схеме), management/commands (синхронизация с HeadHunter, «Работа России» / Trudvsem, DreamJob, служебное обслуживание данных), templatetags, migrations, templates, static.

Схема связывает обзор репозитория с детализацией по файлам внутри приложений и задаёт соответствие подсистем предметной области каталогам проекта – это используется при ревизии архитектуры и планировании доработок.

2.3.5. Функциональная схема

На рисунке 10 представлена функциональная схема web-приложения JobFlex: точки входа, ролевые ветви и общие сервисы. Схема показывает переход от структуры проекта к пользовательским сценариям и потокам взаимодействия.

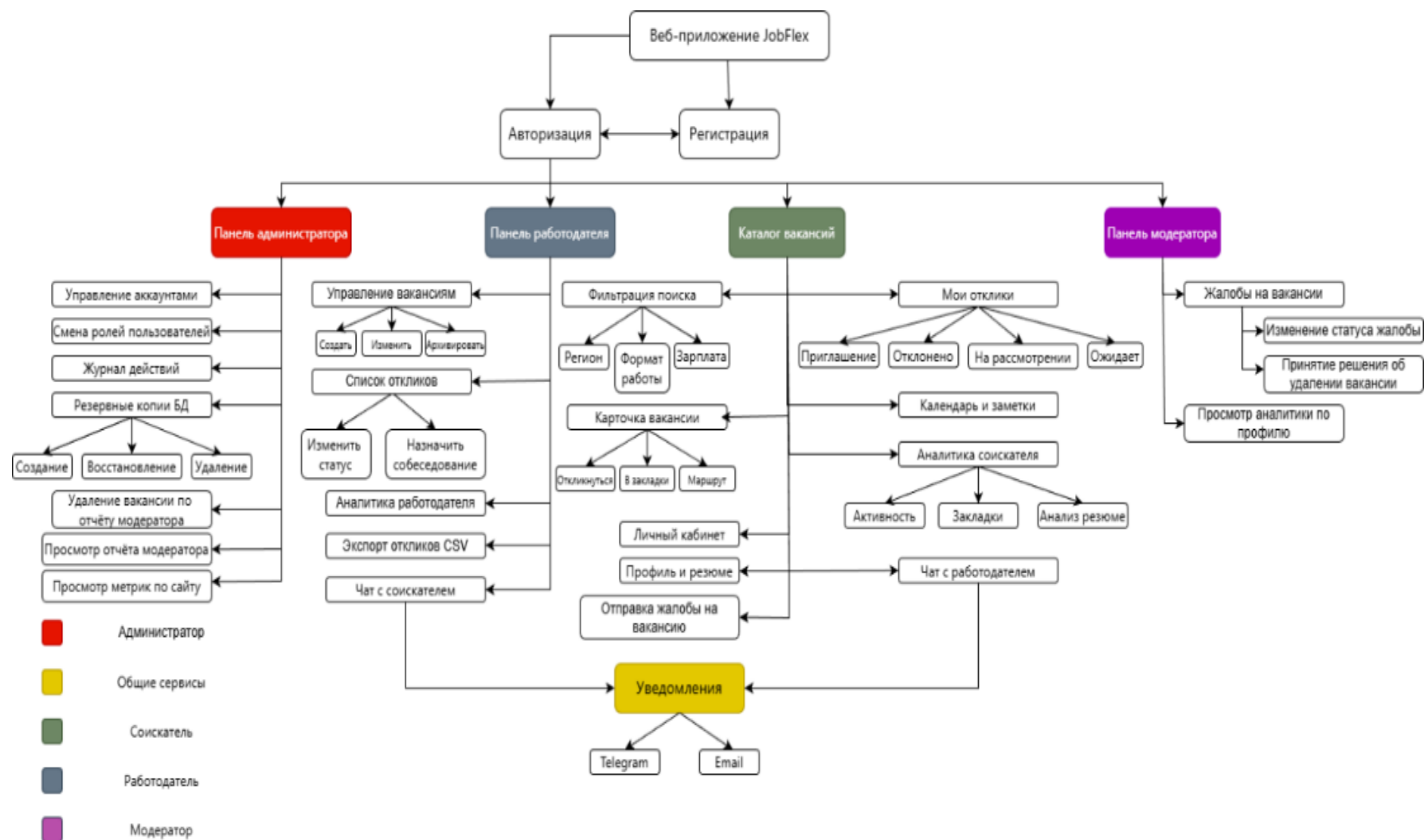


Рисунок 10 – Функциональная схема web-приложения JobFlex

Доступ – через регистрацию и вход; функции зависят от роли (соискатель, работодатель, администратор, модератор).

Соискатель: каталог (фильтры, карточка, отклик, закладки, маршрут на карте); в кабинете – профиль/резюме, отклики, чат, календарь, аналитика, жалобы.

Работодатель: свои вакансии (создание, правки, архивация), отклики и статусы, собеседования, аналитика, CSV, чат.

Администратор: пользователи, журнал действий, бэкапы БД (создание/восстановление/удаление), общие показатели.

Модератор: жалобы, статусы и решения (в т.ч. скрытие вакансий), аналитика модерации.

Уведомления – по e-mail и Telegram при согласии и настройках сервера.

Сквозной процесс: поиск и отклик, обработка работодателем, модерация и администрирование.

2.3.6. Схема тестирования

Схема и методики тестирования представлены в приложении Г «Сценарий и результаты тестовых испытаний»

2.3.7. Схема пользовательского интерфейса

На Рисунке 11 продемонстрирована схема интерфейса web-платформы JobFlex. На данной схеме графически продемонстрированы все окна и переходы между ними.

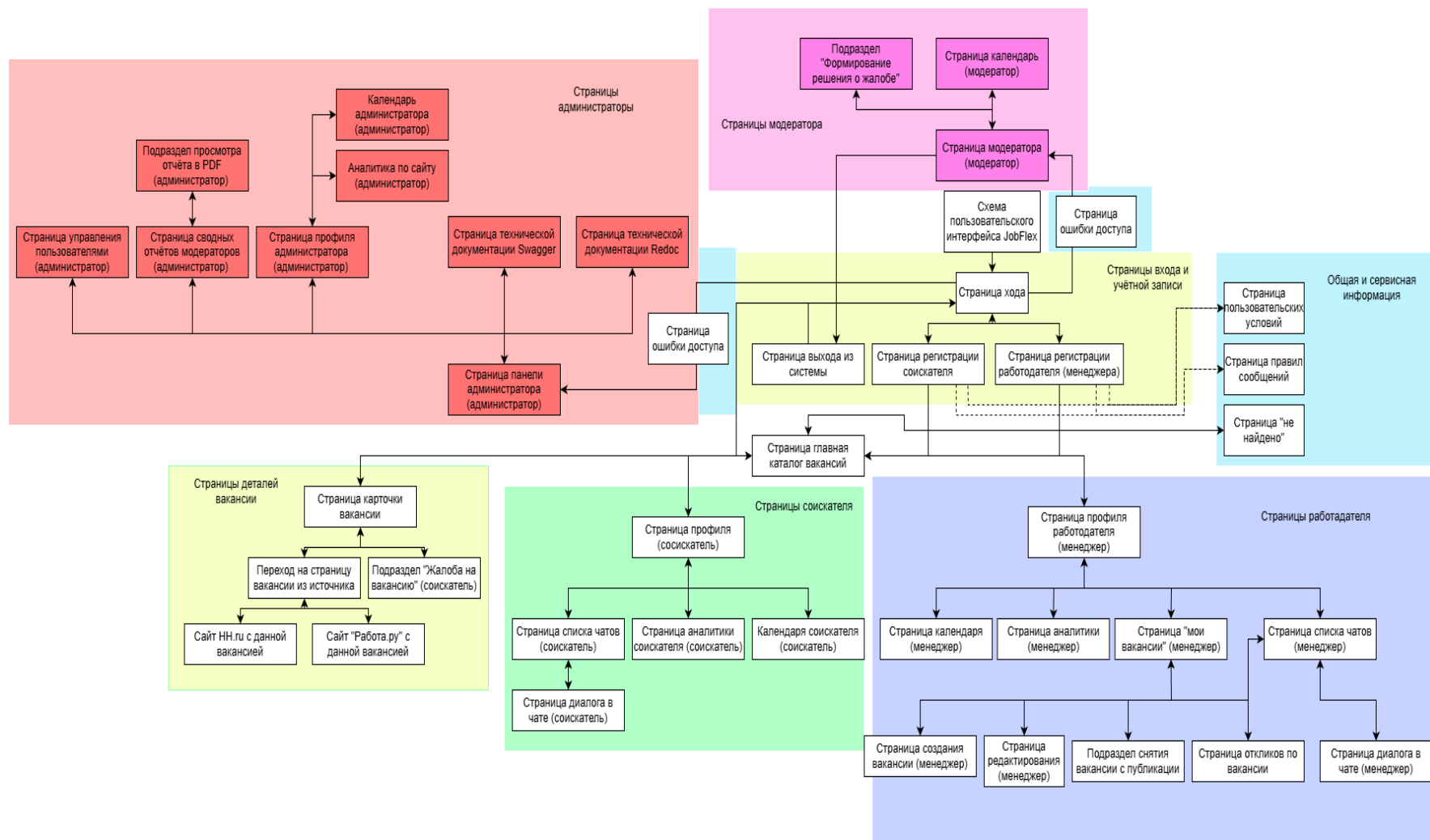


Рисунок 11 – Схема пользовательского интерфейса платформы JobFlex

2.3.8. Диаграмма классов

Диаграмма классов представлена в приложении Ж «Диаграмма классов».

2.4. Результат работы программы

В результате выполнения поставленной задачи разработано программное решение – web-приложение JobFlex: клиент-серверный web-ресурс для агрегирования вакансий из внешних источников (HeadHunter и «Работа России»), на рисунке 12 показан пример интерфейса – карточка вакансии с описанием, атрибутами.

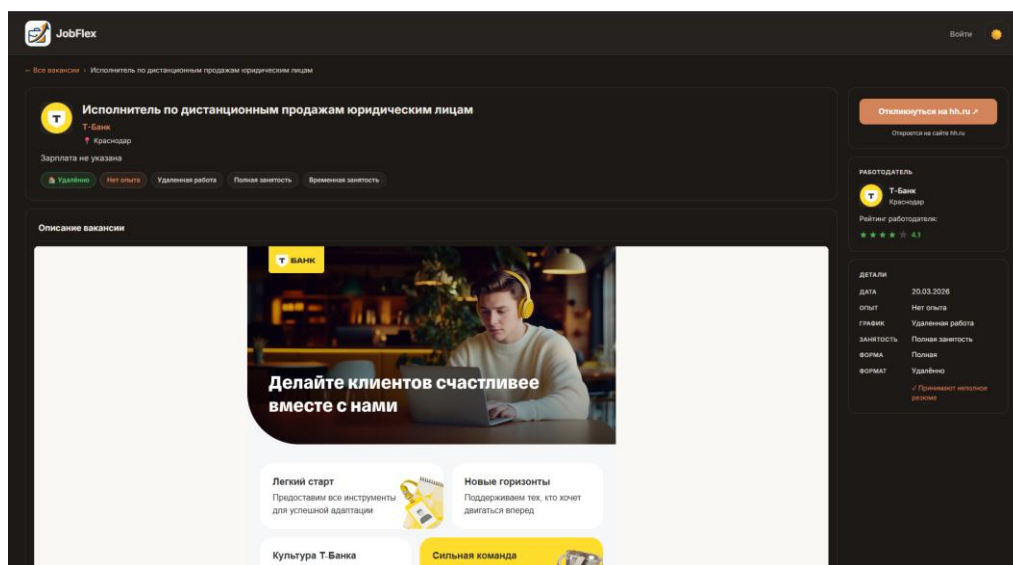


Рисунок 12 – Страница с детальной информацией о вакансии

Решение построено по клиент-серверной архитектуре (см. п. 2.3.1), реализовано разграничение полномочий и функциональных возможностей по ролям (соискатель, работодатель, администратор, модератор) и отражено на функциональной схеме и схеме пользовательского интерфейса (см. п. 2.3.5, 2.3.7).

Структура данных описана в п. 2.3.2 (логическая модель) и п. 2.3.3 (физическая модель). Перечень атрибутов, типов и обязательности полей

пользовательского интерфейса и REST API приведён в приложении Е «Словарь данных» в таблице 1.

Скрипт базы данных помещён в приложение Б «Скрипт базы данных».

Порядок работы пользователя с web-ресурсом приведён в приложении В «Руководство пользователя».

3. ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

3.1. Инструментальные средства разработки

Web-приложение JobFlex реализовано на Python 3.14 с использованием Django и REST API на базе Django REST Framework. Описание и проверка эндпоинтов выполняются по OpenAPI в интерфейсах Swagger UI и ReDoc.

Основная среда разработки – Visual Studio Code (редактирование, навигация по коду, виртуальное окружение, отладка, работа с Git). Запросы к API проверялись в Postman; клиентская часть – в браузере Yandex Browser с инструментами разработчика.

Данные хранятся в SQLite; доступ – через ORM Django, изменения схемы фиксируются миграциями, ограничения целостности и уникальности задаются в моделях и миграциях.

Фоновые операции (синхронизация вакансий с внешними источниками, дозагрузка описаний, напоминания о собеседованиях, рассылки) выполняются в Celery с брокером и бэкендом результатов Redis. Celery Worker и Celery Beat работают отдельно от процесса web-сервера, чтобы длительные задачи не блокировали обработку HTTP-запросов.

Контроль версий – Git; в GitLab настроен CI/CD (линтинг, сборка окружения, прогон тестов по стадиям пайплайна).

Интеграции по HTTP/HTTPS: API hh.ru и opendata.trudvsem.ru («Работа России») – для получения и актуализации вакансий; DreamJob – для сведений о рейтингах работодателей; 2ГИС[12] – для карт и маршрутов; Telegram Bot API[11] – для уведомлений и привязки учётной записи к чату.

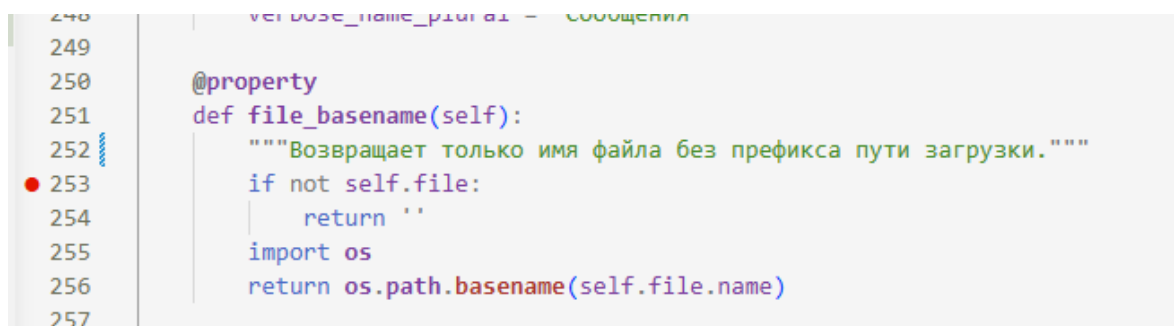
3.2. Отладка программы

Отладка серверной части JobFlex выполнялась в среде Visual Studio Code с использованием встроенного отладчика Python к проекту Django: устанавливались точки останова в представлениях и обработчиках REST

API, просматривались значения переменных и стек вызовов, выполнялось пошаговое исполнение при воспроизведении откликов, импорта вакансий и сценариев администрирования. Дополнительно проверялись запросы к API во внешнем инструменте Postman[20] и журналирование фоновых задач Celery.

В ходе отладки были выявлены и устранены ошибки следующих типов: неверные предположения о структуре ответов внешних API (HeadHunter, «Работа России»); ошибки ветвления при проверке ролей и прав доступа; несогласованность данных при сериализации в JSON; граничные случаи форм (пустые поля, вилка зарплаты); ошибки асинхронных сценариев при обращении к очереди Redis [6]. Количественно на стадии активной отладки было зафиксировано порядка 24 случаев, потребовавших исправления кода (без учёта предупреждений линтера и рефакторинга по замечаниям руководителя).

Пример использования точки останова при отладке представлен на рисунке 13.



```
248
249
250     @property
251     def file_basename(self):
252         """Возвращает только имя файла без префикса пути загрузки."""
253         if not self.file:
254             return ''
255         import os
256         return os.path.basename(self.file.name)
257
```

Рисунок 13 – Точка останова в IDE при отладке серверного кода

3.3. Защитное программирование

На рисунке 14 показан фрагмент кода с try / except и/или использованием transaction.atomic() при выполнении операции, затрагивающей базу данных.

```
def api_profile_documents(request):
    issued_date_val = None
    if issued_date_raw:
        try:
            issued_date_val = date.fromisoformat(issued_date_raw)
        except ValueError:
            return JsonResponse({'error': 'invalid_issued_date', 'detail': 'Некорректная дата выдачи.'}, status=400)
```

Рисунок 14 – Обработка исключений и атомарная транзакция

На рисунке 15 представлен пример сообщения об ошибке в интерфейсе после отправки формы с некорректными данными.

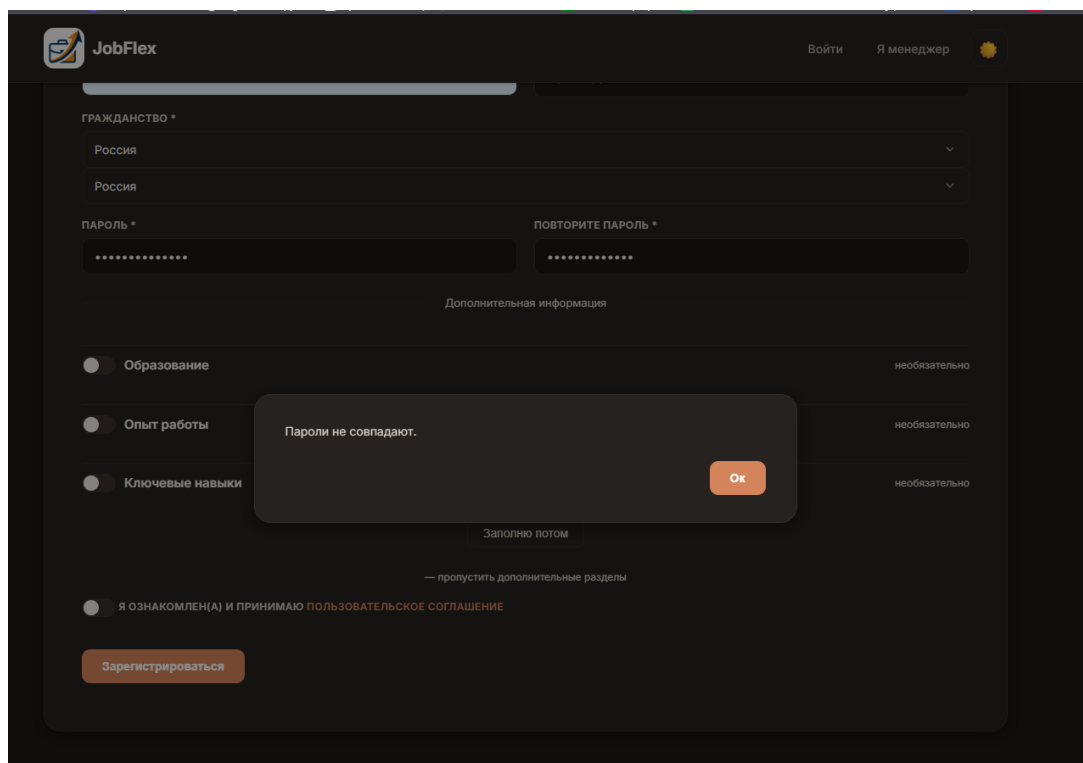


Рисунок 15 – Уведомление пользователя об ошибке валидации

На рисунках 16 и 17 представлен пример защитного программирования на основе отслеживания CSRF сеанса и токена пользователя при попытке зайти на запрещённые страницы для того или иного пользователя.

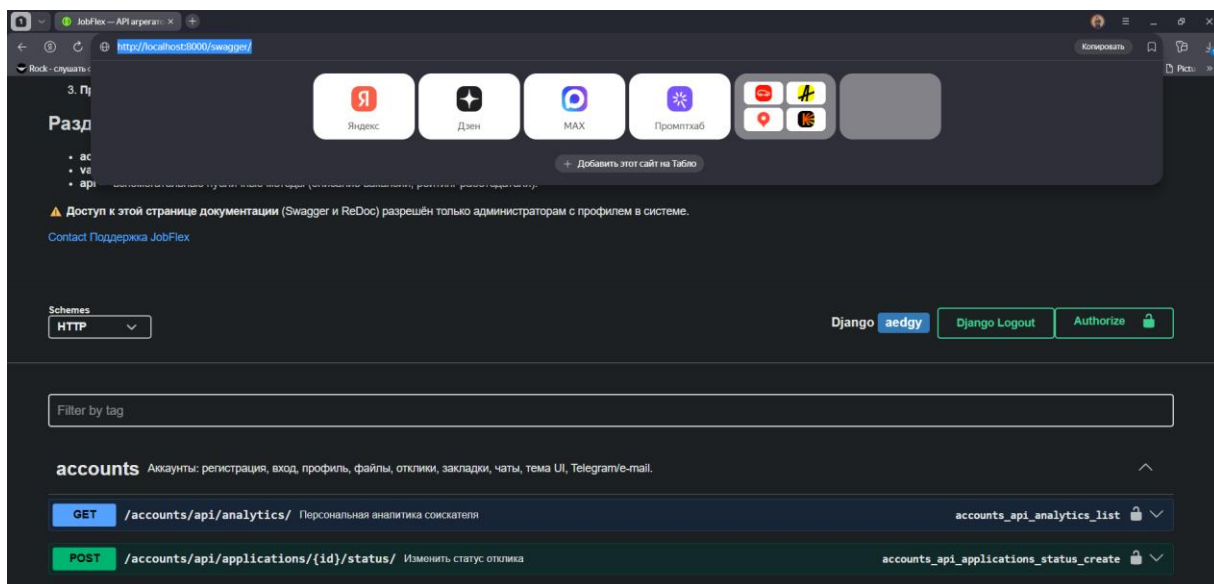


Рисунок 16 – Авторизованный пользователь с ролью администратор переходит на страницу API документации Swagger

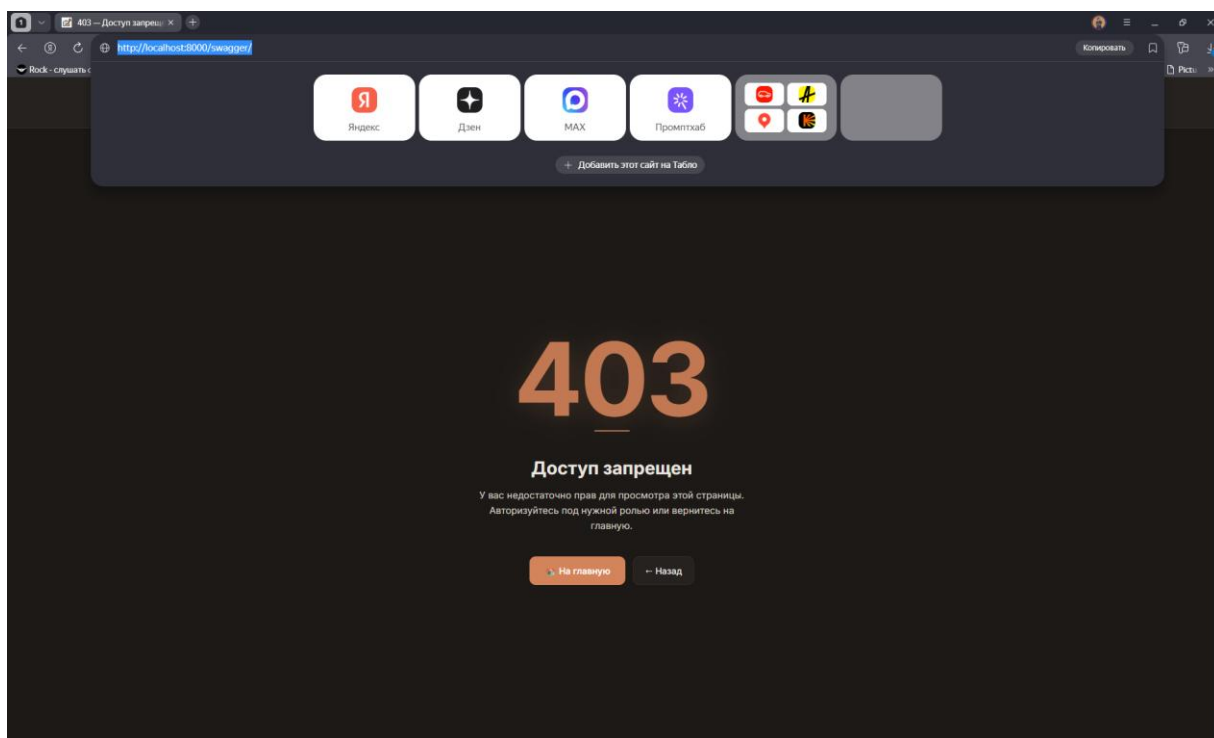


Рисунок 17 – Попытка «гостя» зайти на страницу API документации Swagger

3.4. Характеристики программы

Программный продукт JobFlex представляет собой web-приложение (клиент – браузер, сервер – Django). Ресурс не привязан к одной операционной системе на стороне пользователя: достаточно современного

web-браузера с поддержкой HTML5, CSS3 и JavaScript (актуальные версии Chromium, Firefox, Safari; при разработке дополнительно проверялся Yandex Browser). На стороне сервера используется Python и фреймворк Django с Django REST Framework; для фоновых задач применяются Celery и Redis; данные хранятся в файле SQLite.

Логическая декомпозиция кода на модули (пакеты приложения) отражена на структурной схеме (см. п. 2.3.4) и детализирована в таблице 6.

Таблица 5 – Описание модулей программного комплекса JobFlex

№	Наименование модуля	Описание	Условный размер исходного кода
1	2	3	4
1	config	Настройки Django (settings.py), корневые urls.py, Celery (celery.py), WSGI/ASGI, OpenAPI, проверки состояния сервисов.	1125
2	accounts	Профили (роли), отклики, чаты, собеседования, документы, календарь, UI-настройки, журнал API; страницы и REST; Celery; Telegram; шаблоны и статика приложения.	8425
3	vacancies	Вакансии, закладки, просмотры, жалобы, модерация; каталог, карточка, создание на сайте; REST; импорт HH, Trudvsem, DreamJob; Celery и management-команды; шаблоны и статика.	5443
4	templates (корень)	Общие шаблоны (базовый каркас); подключают accounts и vacancies.	50
5	static / media	Статика (CSS, JS, картинки UI) и загрузки пользователей (аватары, документы, вложения чата).	10 299
6	backups	Резервные копии БД и сопутствующих файлов (админ-сценарии).	389 (см. примечание о пересечении с п. 2)
7	tools	Утилитарные скрипты (справочники, проверки API, подготовка данных); вне рантайма HTTP.	423
8	plugins	Внешние фронтенд-ресурсы (в т.ч. Magic CSS).	4928
9	manage.py, run_all.py, locust.py, unittest.py	manage.py – стандартный CLI Django; run_all.py – при наличии: запуск воркера Celery и Beat на стенде/в разработке. Locust.py – нагрузочное тестирование, unittest.py – U.тесты	539

Суммарный объём исходных текстов на языке Python по репозиторию (без виртуального окружения и крупных сторонних плагинов) составляет порядка 32 тыс. строк; точное значение зависит от состава включённых в измерение каталогов.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы решена задача разработки web-приложения JobFlex – комплекса для агрегирования и поиска вакансий, взаимодействия соискателей и работодателей (отклики, чаты, собеседования, уведомления), администрирования и модерации.

Реализована клиент–серверная архитектура на Django и Django REST Framework с реляционным хранением в SQLite, логической и физической моделями и словарём данных. Фоновые задачи – Celery и Redis; интеграция с HeadHunter, «Работа России» (opendata.trudvsem.ru), а также DreamJob, 2ГИС, Telegram Bot API и e-mail. Интерфейс – адаптивное web-приложение с ролями; API описано в OpenAPI (Swagger и ReDoc).

Практическая значимость: автоматизация сбора и актуализации вакансий, снижение трудозатрат на поиск работы и централизация коммуникаций на одной платформе; журналирование, резервное копирование и средства контроля повышают управляемость системы.

Дальнейшее развитие: при росте данных – выделенная СУБД; расширение источников и аналитики; тесты; контейнеризация контура «веб – воркер – брокер» для промышленного развёртывания.

Цель достигнута: функционал реализован, пользовательские и административные сценарии поддерживаются, проект и данные задокументированы. Результаты пригодны как основа для доработок в кадровых сервисах и интеграции HR-процессов с внешними площадками.

СПИСОК ИСПОЛЬЗУЕМЫХ МАТЕРИАЛОВ

1. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. Введён 1979-01-01.
2. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» (с изм. и доп.) [Электронный ресурс]. – URL: https://www.consultant.ru/document/cons_doc_LAW_61801/ – (Дата обращения: 12.03.2026).
3. Django Documentation [Электронный ресурс] / Django Software Foundation. – URL: <https://docs.djangoproject.com/en/stable/> – (Дата обращения: 28.02.2026).
4. Django REST framework: Documentation [Электронный ресурс]. – URL: <https://www.django-rest-framework.org/> – (Дата обращения: 05.03.2026).
5. Celery Documentation [Электронный ресурс]. – URL: <https://docs.celeryq.dev/en/stable/> – (Дата обращения: 10.03.2026).
6. Redis Documentation [Электронный ресурс] / Redis Ltd. – URL: <https://redis.io/docs/latest/> – (Дата обращения: 10.03.2026).
7. SQLite Documentation [Электронный ресурс] / Hipp W. et al. – URL: <https://www.sqlite.org/docs.html> – (Дата обращения: 01.03.2026).
8. Документация языка Python 3.x [Электронный ресурс] / Python Software Foundation. – URL: <https://docs.python.org/3.14/> – (Дата обращения: 18.03.2026).
9. HeadHunter: Документация для разработчиков / API-партнёрский сервис [Электронный ресурс] / ООО «Эйч Карьера». – URL: <https://dev.hh.ru/> – (Дата обращения: 18.03.2026).
10. Федеральная служба по труду и занятости. Общероссийское агентство баз «Работа в России»: открытые данные (в т.ч. вакансии) и

- документация API на площади opendata.trudvsem.ru [Электронный ресурс]. – URL: <https://opendata.trudvsem.ru> – (Дата обращения: 22.03.2026).
11. Telegram Bot API [Электронный ресурс]. – URL: <https://core.telegram.org/bots/api> – (Дата обращения: 26.03.2026).
 12. Документация 2GIS API [Электронный ресурс] / DualGIS. – URL: <https://docs.2gis.com/> – (Дата обращения: 08.03.2026).
 13. Спецификация OpenAPI (версии 3.x) [Электронный ресурс] / OpenAPI Initiative. – URL: <https://spec.openapis.org/oas/latest.html> – (Дата обращения: 15.03.2026).
 14. Git Documentation [Электронный ресурс]. – URL: <https://git-scm.com/doc> – (Дата обращения: 02.03.2026).
 15. Swagger UI: официальный репозиторий и описание интерфейса OpenAPI-Swagger [Электронный ресурс]. – URL: <https://swagger.io/tools/swagger-ui/> – (Дата обращения: 16.03.2026).
 16. ReDoc: OpenAPI-документирование [Электронный ресурс]. – URL: <https://redocly.com/redoc/> – (Дата обращения: 16.03.2026).
 17. Lyosik. Клиент-серверная архитектура. SA для самых маленьких [Электронный ресурс] // Habr – URL: <https://habr.com/ru/articles/836464/> – (Дата обращения: 11.03.2026).
 18. Ларман К. Применение UML и шаблонов проектирования. 2-е изд. Пер. с англ. – М.: Вильямс, 2006.
 19. Том Пайлот. Практический анализ IDEF0 для начинающих [Электронный ресурс] // ТРИНИОН ИТ. – URL: <https://trinion.org/blog/idef0-znakomstvo-s-notaciyey-i-primer-ispolzovaniya> – (Дата обращения: 25.03.2026).

20. Postman Learning Center – документация по отправке запросов и тестированию API [Электронный ресурс]. – URL: <https://learning.postman.com/docs/introduction/overview/> – (Дата обращения: 20.03.2026).
21. GitLab CI/CD documentation [Электронный ресурс] / GitLab Inc. – URL: <https://docs.gitlab.com/ee/ci/> – (Дата обращения: 28.03.2026).
22. Docker Documentation overview [Электронный ресурс] / Docker Inc. – URL: <https://docs.docker.com/> – (Дата обращения: 05.03.2026).
23. Betterstack Locust Documentation [Электронный ресурс] / Locust contributors. – URL: <https://betterstack.com/community/guides/testing/locust-explained/> – (Дата обращения: 17.05.2026).
24. unittest – Unit testing framework [Электронный ресурс] / Python Software Foundation. – URL: <https://docs.python.org/3/library/unittest.html> – (Дата обращения: 17.05.2026).